

FORECASTS | POLICIES | IMPROVERS | EVIDENCE

FROM FORECASTING MODELS TO SELF-IMPROVING TRADING SYSTEMS

Learning to improve the search,
not only the forecast

A didactic application of the Self-Taught Optimizer (STOP) framework

THE QUESTION THAT ORGANIZES THIS PAPER

How can a fixed language model help us find better forecasting combinations, and then help us improve the procedure that searches for those combinations? This paper follows that question from familiar statistical models to nested improvement loops, interprets the results saved in the companion notebook, and develops a path toward richer trading strategies and coordinated agents.

Alejandro Reynoso

Algorithmic Trading

September 2026

Pedagogical companion paper

Abstract

Students of algorithmic trading usually begin by learning how to construct forecasts, compare models, and translate signals into decisions. Artificial intelligence introduces a further possibility: the research procedure that discovers useful models can itself become an object of optimization. This paper develops that idea through a bounded application of the Self-Taught Optimizer framework. Twenty fixed forecasting methods produce reusable predictions for synthetic markets. An inner loop uses a language model to propose regime-adaptive model combinations and an external numerical evaluator to select among them. An outer loop proposes alternative search configurations and evaluates each by running its inner search. The architecture therefore separates forecasting, proposal generation, measurement, and selection while connecting them through nested feedback.

The discussion explains the implementation in detail without assuming that the reader has opened the notebook. Particular attention is given to chronological information boundaries, the meaning of utility, the difference between a forecasting policy and a trading strategy, and the distinction between improving a configuration and rewriting a program. The stored notebook results offer an instructive outcome: validation performance improves, but held-out performance deteriorates. This divergence becomes a central lesson in the interpretation of self-improvement. The paper then develops possible extensions involving diagnostic feedback, budget-aware experimentation, complete trading strategies, language-model-based strategy agents, and coordinated research organizations. Throughout, more capable adaptation is treated as an experimental proposition requiring evidence, rather than an automatic consequence of adding recursion or more agents.

How to read this paper

The intended reader understands basic returns, regression, and model evaluation but need not have studied agentic systems. The first part builds the conceptual bridge from models to systems. The middle follows the notebook's data, policies, inner loop, and outer loop. The final part interprets the saved run and develops a research agenda. Equations are used sparingly to make a few essential relationships precise; every equation is explained in words. Numerical results identified as "saved run" come from the attached notebook's stored outputs. They have not been regenerated for this paper. Other numerical examples are explicitly illustrative.

The paper describes the complete architecture, inner loop, and outer loop directly in the text. It can therefore be read, compiled, and taught without external image files or an active connection to a language-model service.

Contents

Abstract	2
1 The next step after learning forecasting models	4
2 From an answer to an improver	4
3 A map of the complete architecture	5
4 Why the laboratory begins with synthetic markets	6
5 Time, information, and the meaning of a forecast	7
6 Twenty models as a collection of competing hypotheses	7
7 The economics of precomputation	8
8 A forecasting policy is a conditional combination	9
9 Utility makes improvement an operational claim	10
10 From one market's utility to a policy selection score	11
11 Inside the inner loop: proposing, testing, and retaining	11
12 A worked inner-loop example	12
13 Inside the outer loop: testing the search itself	13
14 Budgets, memory, and the limits of an instruction	14
15 What the saved notebook run actually shows	15
16 Reading the final policy and the diagnostic plots	16
17 Why self-improvement needs independent evidence	16
18 A more informative inner loop	17
19 Making the outer loop more discriminating	18
20 From forecasts to trading strategies	19
21 Strategies represented by LLM-based agents	19
22 Agents as an organized research process	20
23 A progression for classroom experiments	21
24 Conclusion: the discipline of improving improvement	22
A Connecting the narrative to the notebook	23
B Source notes	24
References	24

1 The next step after learning forecasting models

A student who has learned regression, moving averages, decision trees, and neural networks has acquired an important collection of tools. Yet a practical forecasting exercise rarely ends with knowing how each tool works. The researcher must still decide which tools deserve attention, how to compare them fairly, when to combine them, and what to do when market conditions change. Those decisions form a research process. In a conventional exercise, the process is largely supplied by the instructor or designed manually by the researcher. The model is optimized, while the procedure surrounding the model is treated as fixed.

This notebook moves that surrounding procedure into the foreground. Its starting question is familiar: can a collection of forecasts be combined more effectively than a simple initial ensemble? Its second question is less familiar: can the system discover a more effective way of searching for those combinations? Answering the second question requires a change in the unit of analysis. Instead of looking only at a statistical estimator, we look at a system containing estimators, a language model, numerical tools, representations, selection rules, and a record of previous decisions. Performance becomes a property of the interaction among these components.

Consider a research assistant asked to improve a forecast. The assistant could replace a linear model with a more complex algorithm. Alternatively, the assistant could discover that the existing models were evaluated on inconsistent periods, that several forecasts duplicate one another, or that a modest ensemble performs more reliably than a single elaborate model. The latter improvements arise from organizing research more effectively. The intelligence of the underlying forecasting algorithms need not increase for the resulting system to become more useful. The notebook gives this organizational possibility a form that students can execute and inspect.

The language model contributes proposals rather than a guarantee of correctness. It reads a compact description of the available models, a current combination, and performance information. It proposes alternatives that the program can evaluate. A candidate's survival depends on its measured utility. This makes the application different from asking a chatbot which forecasting method is best and accepting the answer. The relevant output is an experimentally selected configuration, accompanied by a numerical score and a trace of how it was discovered.

For algorithmic trading, this is a useful intermediate step. The notebook does not yet trade, allocate live capital, or simulate transaction costs. It addresses a research problem that sits upstream of those activities: selecting and combining forecasting methods. Keeping that boundary clear allows students to understand the architecture before introducing the additional complications of positions, execution, and portfolio risk. The eventual extension to trading is substantial, but the core learning cycle can already be studied in this simpler setting.

2 From an answer to an improver

The supplied general teaching note introduces STOP through the distinction between producing an answer and organizing a search for better answers [2]. A single language-model response is one candidate. A procedure that generates several candidates, evaluates them, and returns a selected result is an improver. Once that procedure exists, its design becomes another legitimate object of investigation. How many alternatives should it request? Should it retain only one promising solution or several? Should later requests explore new directions or refine earlier proposals? These are design choices with measurable consequences.

The original STOP research studies a language-model-infused program that improves programs and can also improve the scaffold organizing its own use. The language model itself is not retrained during that process. This is the central connection to the present application: effective capability can change through the organization of generation, evaluation, and search around a fixed model [1]. The attached teaching note develops the analogy of a research laboratory whose methods improve even when its scientists remain the same people. We retain that analogy because it separates the quality of a component from the quality of the institution using it.

Imagine two laboratories with identical forecasting models and access to the same language model. The first laboratory asks for one proposal and immediately adopts it. The second requests several proposals, checks their admissibility, evaluates them against a baseline, preserves strong alternatives, and investigates further only within a defined budget. Differences in their outcomes would reflect their procedures, not necessarily differences in their component models. The outer loop treats the design of that procedure as a testable hypothesis. A search recipe must demonstrate its value through the policies it actually finds.

The present notebook deliberately narrows the original idea. It does not ask the LLM to return new executable Python code for the improver. Instead, it asks for values in a predefined configuration: candidate count, search rounds, survivor count, and strategic instructions. Fixed Python interprets those values. We should therefore describe the application as bounded, STOP-inspired search-configuration improvement. It preserves the nesting of solution search and improver evaluation, but it is not a reproduction of unrestricted program-level self-modification. That distinction is part of the lesson, not a defect to conceal.

A bounded representation makes the experiment easier to teach. Students can identify every permitted change and inspect how it affects execution. At the same time, they can see the limitation: an instruction describing a sophisticated method has operational consequences only where the fixed program provides the necessary capability. A system may be asked to preserve diversity or stress-test candidates, but prose alone cannot create a diversity constraint or a stress-testing engine. The configuration is an interface to existing behavior, and understanding that interface is essential to understanding the system.

3 A map of the complete architecture

The architecture contains several distinct objects that should not be collapsed into the word “model.” The synthetic environment generates return histories. The base models transform historical information into forecasts. A forecasting policy selects model identifiers and weights for observable market conditions. An improver searches for policies. Finally, the outer procedure searches for improver configurations. Each object answers a different question, and each has a different relationship to the evaluation data.

The environment is prepared first, followed by a table of forecasts from the twenty base models. These tables are the numerical substrate of subsequent experiments. The policy does not refit the models each time it is evaluated; it selects and combines columns from the tables. The inner improver asks the LLM to suggest alternative policies, scores those policies on validation observations from training markets, and retains the strongest candidates. Its output is a policy, a score, and a history of inner rounds. The outer loop uses that inner computation as its own experiment.

An improver’s score is inherited from the policy it discovers. This inheritance explains the nesting. To evaluate a forecasting policy, the program must construct predictions and compare them with outcomes. To evaluate an improver, it must first run a search that produces such a policy. The outer evaluator cannot determine search quality merely by reading an attractive description of the search.

It has to pay the computational cost of using that procedure and observing the result.

The final evaluation is separate. Policies selected during the training-market search are applied to test periods from additional synthetic markets. This allows the notebook to ask whether a selected configuration continues to work beyond the environments used for selection. It does not establish that the improver itself can successfully discover new policies on unfamiliar tasks, because the held-out stage does not rerun the inner search there. This distinction between policy transfer and improver transfer will become important when interpreting the results.

The architecture should be understood as a hierarchy of responsibilities rather than a collection of interchangeable AI components. Numerical forecasting happens in one part of the architecture, candidate generation in another, and acceptance in a third. Information flows between them through explicit representations. Synthetic markets feed a fixed forecasting menu; an inner loop searches for regime-adaptive policies; an outer loop compares search configurations; and selected policies are subsequently evaluated on held-out markets. This separation allows a student to ask where an observed gain came from: a model, a combination, a search setting, a changed data protocol, or simply a favorable evaluation sample.

4 Why the laboratory begins with synthetic markets

The notebook generates seven synthetic index histories, each beginning with 900 observations before feature construction. Four market seeds are designated for search and three for held-out evaluation. Different seeds create different realized histories under the same generator. The generator contains four hidden states: a relatively calm rising market, a volatile declining market, a sideways mean-reverting market, and a recovery state with momentum. State transitions are persistent but uncertain. Within each state, returns reflect drift, volatility, dependence on a previous return, slowly moving factors, and random shocks.

This construction gives familiar forecasting methods something meaningful to disagree about. A model emphasizing recent movement may behave differently from one emphasizing a historical average. A robust estimator may react differently to large observations than a least-squares estimator. A flexible nonlinear model may capture some relationships but also estimate unstable patterns. The experiment does not assume that every complicated model will perform well. Indeed, a useful synthetic environment can teach precisely the opposite: a weak signal embedded in substantial noise may be difficult to forecast even when its generating mechanism has structure.

The hidden state is retained for diagnostics but excluded from the forecasting feature list. Students should imagine the generator as an instructor who knows how the exercise was constructed, while the forecasting system sees only the permitted observations. If the system were given the true regime, it would receive information that is generally unavailable in a real forecasting problem. Keeping the hidden state outside the decision interface makes regime adaptation an inference problem rather than an oracle lookup.

Synthetic markets offer control, repeatability, and low-cost experimentation. They make it possible to change transition persistence or shock intensity in a later version and ask whether the architecture still behaves sensibly. However, several seeds from one generator are not equivalent to several independent real-world market mechanisms. Every history shares structural assumptions about the return process. A policy may transfer across seeds while remaining highly specialized to those assumptions. The current test therefore measures transfer within a synthetic family, not readiness for a real exchange.

The educational use of simulation also changes how we interpret failure. If a policy cannot beat a

simple baseline, that result need not imply that the program failed to run. It may indicate that the permitted signals are weak, that the selected models are poorly calibrated, or that the search did not discover a useful combination. These possibilities can be investigated separately because the environment is controlled. The synthetic laboratory is valuable because it exposes those questions, not because it promises a profitable result.

5 Time, information, and the meaning of a forecast

The forecasting task is to predict the next day's log return using information available at the current observation. A log return is consistent with the generator's price update: today's index level is multiplied by the exponential of the realized return to obtain the next level. This convention matters later when a return forecast is converted into a price forecast. Students do not need advanced stochastic calculus to follow the exercise, but they do need to know what quantity the models predict and when its inputs become observable.

Feature construction creates lagged returns, rolling return means, rolling volatilities, moving-average price ratios, and lagged synthetic macroeconomic, valuation, and volume variables. The target column shifts the return series forward by one observation. Consequently, the target attached to row t is the return to be predicted, while the target attached to row $t - 1$ has already been realized by the decision time at row t . This explains why the historical target column can legitimately appear in model training and in the observable regime proxy. The word "target" in a column name does not by itself imply future leakage; the row alignment determines whether the value is known.

The code removes rows with missing values after constructing rolling features and the shifted target. With the supplied 900-observation history, the 60-observation rolling window and the missing final target leave 840 feature rows. The boundaries 560 and 730 refer to positions in this cleaned feature table, not the corresponding positions in the original raw history. Validation covers 170 forecast rows, from position 560 through 729. Testing covers the remaining 110 rows, from 730 through 839. These counts explain the experiment more accurately than treating 900 as the number of usable supervised observations.

The forecasting engine uses an expanding historical sample. At each scheduled machine-learning refit, it trains only on rows before the refit anchor. It then applies the fitted model to feature vectors in the following block. Statistical models retain daily updating or fitting. This protocol allows earlier realized observations to enter later model fits, including observations from earlier parts of a validation or test period. That is ordinary walk-forward adaptation of the base forecaster. It is different from using later outcomes to choose the forecasting policy before those outcomes occur.

The assumed decision time is after the current observation's relevant information is available. A future intraday implementation would need to replace that broad assumption with explicit timestamps for prices, releases, orders, and fills. Even a correct chronological split can be misleading if a feature became available only after the proposed trading decision. The present notebook avoids much of this complexity by using a synthetic daily setting, but it establishes the habit students should carry forward: every forecasting claim has an information boundary as well as a numerical formula.

6 Twenty models as a collection of competing hypotheses

The base menu begins with a zero-return forecast. Although apparently trivial, this is an essential reference: if tomorrow's return is difficult to predict, forecasting no change in log price can be a

demanding benchmark. The next models include the full-history mean, short and longer rolling means, an exponentially weighted average, autoregressions, and exponential smoothing. These methods express different beliefs about persistence, local information, and how quickly the distant past should lose influence. They are transparent enough to help students reason about why an ensemble might behave differently across regimes.

The feature-based models extend the menu through linear regression, ridge, lasso, elastic net, robust regression, nearest neighbors, a decision tree, random forest, extra trees, gradient boosting, support vector regression, and a small multilayer perceptron. These are not twenty independently tuned research programs. They are twenty fixed implementations, with specified settings, applied under a common forecasting protocol. A poor score for one entry is evidence about that implementation in this experiment, not a verdict on an entire family of statistical methods.

For example, regularization changes the degree to which a linear model can respond to weak features. Tree depth and minimum leaf size limit flexibility in the tree-based methods. Scaling is included in several pipelines so that variables with different magnitudes do not dominate merely because of their units. Nevertheless, numerical preprocessing and default optimization criteria can interact awkwardly with very small return targets. If a neural-network implementation performs poorly, the appropriate next step is to inspect its predictions, convergence behavior, and calibration. It would be premature to infer that neural networks cannot forecast financial returns.

The menu also contains fallback behavior. Several forecasting routines return a recent mean if fitting fails. This keeps the pedagogical experiment running, but it changes the meaning of the named model on those observations. A column labelled as an autoregression can partly or entirely reflect its fallback. The saved scorecard shows identical aggregate scores for AR(1), AR(5), and the rolling mean over twenty observations. That equality is a reason to inspect forecast columns and fitting behavior; it is not, by itself, proof of the cause. A later version should record fallback frequency with the same care as predictive accuracy.

Thinking of models as hypotheses rather than brands encourages better research practice. The relevant question is not which model name sounds most sophisticated. It is what pattern an implementation can represent, what information it receives, how its errors relate to those of other candidates, and whether its contribution survives evaluation. The inner loop searches over combinations of these hypotheses. Its success therefore depends partly on the quality and diversity of the menu it has been given. Search cannot reliably recover information that none of its available components capture.

7 The economics of precomputation

A deterministic computation is reproducible under specified conditions; it is not necessarily cheap. Rebuilding a machine-learning model for every forecast date can involve thousands of fits even in a small teaching example. The notebook separates the expensive creation of forecast tables from the cheaper comparison of policies. Once those tables exist, a proposed ensemble can be evaluated by selecting columns, calculating weighted averages, and measuring error. This makes repeated inner searches practical and prevents the outer loop from needlessly recreating the same numerical evidence.

The faster implementation refits the twelve feature-based models every sixty forecast rows while continuing to produce a prediction for every row. Its coefficients remain fixed within a block, but its input feature vector changes each day. Validation and test matrices are constructed separately, so their refit schedules start at their respective boundaries. With 170 validation rows and 110 test rows, there are three validation refits and two test refits for each feature-based model on each market. Across twelve such models and seven markets, that is 420 scheduled fits. Daily refitting would require

23,520 fits for the same 280 forecast rows per market.

This reduction is a change in the modeling protocol, not merely a faster way to calculate identical forecasts. Models update their estimated relationships less frequently. Consequently, a comparison between policies is meaningful only when they use tables prepared under the same schedule. Setting the interval to one restores the daily machine-learning refit schedule, but it also changes the cost of preparing the experiment. The statistical methods still perform their own daily updates or fits, so the number 420 describes the feature-based fits rather than every numerical operation in the notebook.

The forecast cache records a signature incorporating data, feature order, split boundaries, refit interval, relevant code, and numerical-library versions. This is important because an old table should not silently survive a change in the experiment's definition. If the inputs to the forecast process change, reusing the previous predictions can create a false sense that the new experiment has been evaluated. The supplied cache design therefore connects efficiency with provenance: reuse is justified by evidence that the relevant computation is the same.

There is also a separate in-memory cache for language-model responses. Identical model, system-message, and prompt content can reuse an earlier response during the session. This reduces repeated requests and can stabilize repeated evaluations in that session, but it does not make all future runs of the LLM deterministic. Restarting the notebook removes that response cache unless additional persistence is implemented. A complete reproducibility record would retain responses, configurations, environment information, and numerical outputs together. The lesson is that inexpensive iteration should be built on controlled reuse, not on uncertainty about what was actually recomputed.

8 A forecasting policy is a conditional combination

The policy is the central object improved by the inner loop. It contains five blocks: a global fallback and four blocks associated with observable regime proxies. Each block lists between one and six model identifiers and their nonnegative weights. The program normalizes the weights so that their sum is one. Thus, within a selected block, the forecast is a convex combination of the component predictions. It does not use negative model weights, create new model architectures, or choose an arbitrary mathematical function of the prediction table.

Let g_t denote the regime proxy inferred at decision time t , and let $\hat{r}_{t+1}^{(j)}$ denote model j 's forecast. If $J(g_t)$ is the set of model identifiers selected for that proxy, then the policy forecast is

$$\hat{r}_{t+1}^{(\pi)} = \sum_{j \in J(g_t)} w_{j,g_t} \hat{r}_{t+1}^{(j)}, \quad w_{j,g_t} \geq 0, \quad \sum_{j \in J(g_t)} w_{j,g_t} = 1. \quad (1)$$

The equation says that the policy first chooses the relevant block and then averages its model forecasts with the specified weights. The policy can change its mixture when the proxy changes, even though the policy object itself has not been rewritten. Conditional behavior is therefore different from a new round of optimization.

The proxy uses the twenty preceding realized target returns. High volatility takes precedence when their standard deviation exceeds 0.014. Otherwise, a mean above 0.0015 selects positive trend, a mean below -0.0015 selects negative trend, and the remaining observations are classified as sideways. These thresholds are fixed in the supplied implementation. Neither loop is authorized by its schema to change them. The proxy is a coarse observable classification, not an estimate that reveals the generator's true state with certainty.

The global block provides a fallback in the block-selection function. However, the current proxy

function always returns one of the four recognized labels, so normal evaluation uses those four blocks rather than the global one. This creates an instructive subtlety: two serialized policies can differ in their global block while producing identical forecasts in the present experiment. Deduplication by serialized content does not necessarily imply deduplication by behavior. A later search engine could recognize such functional equivalence and avoid spending effort on differences that have no effect on the score.

The initial policy uses equal weights within each block and includes several model families. It is a starting hypothesis, not a privileged optimum. The system measures its score before generating alternatives, allowing students to separate improvement over that starting point from improvement over the zero-return baseline. Those are different comparisons. A search can repair a weak initial ensemble and still fail to deliver useful predictive information relative to the simplest available forecast.

9 Utility makes improvement an operational claim

The word utility can suggest investor preferences, expected wealth, or risk aversion. In this notebook it has a narrower meaning: a numerical forecasting score. The evaluator measures the candidate's root mean squared prediction error and compares it with the error of the zero-return baseline over the same observations. Root mean squared error, or RMSE, squares each forecast error, averages those squared errors, and takes the square root. Large mistakes receive substantial influence because the errors are squared before averaging.

For a market m and a candidate policy π , the implemented score can be written, for ordinary positive error values, as

$$U_m(\pi) = \text{clip}\left(\frac{\text{RMSE}_m(\text{zero return})}{\text{RMSE}_m(\pi)}, 0, 1.5\right). \quad (2)$$

A candidate whose error matches the baseline receives one. A candidate with a larger error receives less than one. A smaller error produces a score above one, up to the cap of 1.5. The implementation separately assigns 1.5 when the candidate error is effectively zero. This special case avoids an unstable division and makes the scoring convention explicit.

As an illustrative calculation, suppose the baseline error is 1.00 percent and a candidate's error is 0.80 percent. Its utility is 1.25. If a different candidate has an error of 1.10 percent, its utility is approximately 0.909. These are examples explaining the formula, not results from the saved run. The score is a ratio, not a percentage return. A value of 1.25 does not mean that an investor earned 25 percent or that the strategy has a particular Sharpe ratio.

The cap creates a deliberate trade-off. It limits the influence of exceptionally favorable ratios, which may help keep the search numerically stable. It also makes all uncapped ratios at or above 1.5 indistinguishable to the selection rule. A system cannot prefer one of those policies on the basis of this score alone. More generally, every detail of the utility function tells the optimizer what differences matter and which differences it can ignore. The evaluator is therefore part of the substantive research design, not a neutral reporting device.

Directional accuracy is also reported, but it is not the quantity used to rank policies. Its implementation compares the signs of realized and predicted returns exactly. A zero forecast has sign zero, so it does not receive credit for predicting the sign of a nonzero return. This means the zero-return baseline can have strong RMSE and poor directional accuracy without contradiction. Different metrics reward different behavior. Students should decide which objective is appropriate before interpreting a result, rather than selecting whichever metric makes the latest candidate look attractive.

10 From one market's utility to a policy selection score

The notebook evaluates each policy on the validation observations of four training markets and averages the four utility values. If $M = 4$, the selection score is

$$\bar{U}(\pi) = \frac{1}{M} \sum_{m=1}^M U_m(\pi). \quad (3)$$

The code calls this function `policy_meta_utility`. At this level, “meta” refers to aggregation across markets. Later, an improver inherits the aggregate score of the policy it finds. Keeping these two uses distinct avoids a common misunderstanding: there is no separate mysterious measure of intelligence added by the outer loop. The numerical foundation remains the measured quality of forecasts.

Averaging ratios gives each market equal weight after its candidate error has been compared with its own baseline. It is not the same operation as pooling all observations, calculating a single overall RMSE, and taking a ratio at the end. This distinction matters when markets have different volatility levels or different error distributions. The current rule rewards average relative performance across the selected environments. It does not guarantee that every environment improves, nor does it explicitly penalize instability between them.

The base-model scorecard uses the same principle to rank the twenty individual methods. The LLM receives the complete model menu but only the top twelve scorecard entries as contextual performance evidence. That scorecard is fixed during the policy search. It can guide proposals toward promising components, but it does not describe the interaction of errors between pairs of models. An individually weaker forecaster could still contribute to an ensemble if its mistakes offset those of another component. Conversely, several highly ranked models may add little diversification if their errors are nearly identical.

This is one reason the search problem remains nontrivial after the individual models have been scored. An ensemble's error depends on how forecasts combine on particular observations, not simply on an average of the models' standalone rankings. Regime-dependent blocks add another layer of conditionality. A component with mediocre overall performance could be helpful within a subset of observations. Nevertheless, the present prompts do not provide a detailed regime-by-model error decomposition. A future version could supply that evidence explicitly and then test whether the additional information improves search outcomes.

The objective is also narrower than robustness in the strongest sense. Average validation utility can rise even if one market deteriorates. A researcher concerned about worst-case performance could add a constraint on minimum market utility or introduce a penalty for dispersion. Such a change would define a new experiment and might select a different policy. The existing notebook offers a clear initial objective. Its pedagogical value comes from making that objective visible enough to question, extend, and compare with alternatives.

11 Inside the inner loop: proposing, testing, and retaining

The inner loop begins by evaluating the initial policy and placing it in a population together with its score. A population is simply the collection of policies still eligible to become parents. During a round, the program sends each surviving parent to the LLM with the model menu, scorecard, requested candidate count, and search instructions. The response is a JSON array of proposed policy objects. It is neither an essay defending a preferred model nor new Python source code. This structured response

gives the numerical program a compact object it can interpret.

Each proposed block contains model identifiers and weights. The parsing layer attempts to recover JSON even if the response includes surrounding text. The policy construction functions remove invalid identifiers, limit block size, and normalize weights. If parsing fails at the proposal level, the parent can be returned as a fallback. This permits the experiment to continue without accepting an unmeasurable answer. However, these convenience checks should not be confused with comprehensive production-grade schema validation. A more demanding implementation would explicitly reject nonfinite values, record all repairs, and verify that identifier-weight alignment remains unambiguous after cleaning.

Python then evaluates each candidate on the same validation tables. Existing parents are added to the candidate pool rather than discarded automatically. After deduplication, the pool is sorted by aggregate utility and truncated to the configured number of survivors. Retaining parents is a form of elitist selection: a new proposal does not replace an existing policy merely because it is newer. Within a fixed inner run, with the same deterministic evaluator and retained parents, the best observed score cannot decrease as the pool is expanded and selected. That local property does not imply improving performance on unseen data.

If more rounds remain, the surviving policies become the next parents. The loop ends after the configured round count, returning the best policy, its score, and a history containing the best score and population size by round. It does not return a guarantee that all useful combinations were explored. The number of possible subsets and weights is much larger than the small collection of candidates tested. The correct description is therefore “the best policy found in this search,” rather than “the best possible combination of forecasting models.”

Feedback currently reaches the LLM in a limited form. Selection changes the parent policy that is supplied in a later round, but the prompt does not include a detailed history of the previous candidate’s errors or the whole population’s results. The numerical system knows which parent survived; the LLM sees that parent’s configuration and the fixed base-model scorecard. The LLM returns model identifiers and weights; Python constructs forecasts, evaluates validation utility, and retains top-scoring parents and candidates. The loop returns the best policy found, its score, and its round history. This is already a feedback loop, but it leaves considerable room for richer evidence-driven revision. Its utility measures relative forecast error, not trading profit.

12 A worked inner-loop example

A small example makes the mechanics easier to follow. Suppose a current high-volatility block contains a zero forecast, a robust regression, and a rolling mean. Its normalized weights are 0.50, 0.30, and 0.20. On one observation, suppose those models predict log returns of 0.00 percent, -0.40 percent, and -0.10 percent. The combined prediction is -0.14 percent. That number is the policy’s forecast for that observation, not its utility. Utility can be calculated only after comparing a sequence of predictions with the corresponding realized outcomes.

Now suppose the LLM proposes a second policy that increases the robust regression’s weight and introduces a different component. Python does not ask whether the proposal sounds more defensible in a volatile market. It constructs forecasts for every validation observation, choosing the relevant regime block at each row, and compares the resulting return series with actual returns. The same policy is tested in each of the four training markets. Its score is the average of the four market utilities. All figures in this example are illustrative; the process, rather than the particular numbers, is the object of interest.

Assume the parent has aggregate utility 0.990, candidate A has 0.997, and candidate B has 0.984. With one survivor, A becomes the next parent. With two survivors, A and the original parent survive. The difference matters in the following round. Retaining two parents permits further proposals around two starting points rather than committing entirely to A. This resembles a small beam search. Yet top-score retention alone does not guarantee meaningful diversity: the two survivors could have nearly identical weights or strongly correlated predictions.

Suppose a later candidate reaches 1.002 on validation. It would beat the zero-return baseline on average in that selection sample, but the practical interpretation would still require caution. The gain could be small relative to sampling variation, concentrated in one market, or associated with errors that are unattractive for a particular trading rule. The notebook’s evaluator identifies a winner under a stated criterion. The researcher remains responsible for determining what that criterion establishes and which additional tests are needed for the next stage of the project.

Finally, the selected policy can be used without calling the LLM again for every forecast in the supplied implementation. The model predictions are available, the proxy function is fixed, and the policy specifies the weights. This distinguishes the cost of discovering a policy from the cost of applying it. A sophisticated search may produce a simple operational rule. Students should recognize this as an important design option: AI can participate in research and configuration while the repeated numerical decision remains transparent and inexpensive.

13 Inside the outer loop: testing the search itself

The outer loop takes an improver specification as its parent. The seed asks for three candidates per round, one round, and one survivor, together with instructions to explore diverse model families. The outer LLM call proposes three distinct alternative specifications. Each has the same four fields as the parent. Candidate counts are bounded from two to five, rounds from one to three, and survivor counts from one to three. The search-style text is limited to five hundred characters by the parsing function. The parent is retained among the specifications to be evaluated.

These fields have different operational roles. Candidate count, rounds, and survivor count are read directly by Python and change the number or structure of inner-loop operations. Search style is passed as language to the inner LLM. It can influence proposals but cannot override the evaluator, expose hidden regimes, change numerical thresholds, or create new tools. A phrase such as “explore then exploit” may encourage a useful pattern of proposals. It does not automatically implement a phase-aware algorithm, because the current inner prompt does not explicitly provide the round number or a complete record of earlier experiments.

Every candidate improver is evaluated by constructing a `ForecastImprover` and running its inner search on the training markets. Each trial starts from the same initial forecasting policy. The outer procedure therefore compares alternative search recipes under a common initial condition; it does not begin every new improver from the best policy discovered by the previous generation. This choice makes the evaluation of the recipe conceptually cleaner, although different recipes can still receive different amounts of computation through their candidate and round settings.

If I_s denotes the inner search governed by specification s , and π_0 the initial policy, then the outer score is

$$V(s) = \overline{U}(I_s(\pi_0)). \quad (4)$$

In words, run the improver, take the policy it returns, and assign that policy’s aggregate validation utility to the improver. The outer loop sorts these scores and returns the highest-scoring specification

together with the candidate table. The evaluation concerns what the recipe achieves in use, not how elaborate its written description appears.

The winning specification becomes the next parent. With depth two, the notebook records generations I_0 , I_1 , and I_2 , representing the seed and two outer selection steps. These labels need not identify different configurations: a parent can win again. The final output includes specifications, discovered policies, generation history, and outer candidate tables. Each proposed recipe is evaluated by actually running a complete inner search from the common initial policy. The winner inherits the score of the best policy it discovers and becomes the next parent. The LLM and executable Python remain fixed; only the allowed configuration fields evolve. This nesting makes the outer layer an experiment about experiments.

14 Budgets, memory, and the limits of an instruction

A search recipe with more rounds and more surviving parents may generate more opportunities to find a high score. That can be useful, but it complicates comparisons. If one recipe receives many more model calls than another, better performance may reflect a larger search budget rather than a more efficient method. The current outer prompt asks the LLM to avoid needless cost, but the numerical ranking uses validation utility alone. There is no subtraction for tokens, elapsed time, or model calls. Cost awareness is an instruction, not an explicit term in the objective.

The notebook sets a maximum of fifty counted successful API calls. The counter is incremented after a successful response, and exact response-cache hits do not consume another call. The implementation also retries failures. A counted-call budget is therefore not identical to a precise billable-token limit or a complete count of attempted network requests. These differences need not obstruct the teaching exercise, but they matter when moving toward operational research systems. A richer budget controller would record attempts, tokens, latency, and numerical evaluation cost separately.

Memory in the present system is also specific. The population retains current survivors. The generation history records selected configurations and scores. The response cache stores text associated with repeated prompts. The audit log records successful calls and outer selections. None of these automatically constitutes a research memory that understands why an experiment failed. Such a memory would require structured records connecting a hypothesis, its implementation, its evaluation conditions, its errors, and the decision taken. This distinction prevents students from assuming that storing outputs is equivalent to learning from them.

The saved outer proposals make the boundary especially visible. Some search-style text refers to stress testing, preserving particular families among survivors, modifying regime-detection sensitivity, or consulting an additional held-out regime. The supplied schema and evaluator do not implement those actions. Survivor selection remains a score-based truncation, regime thresholds remain fixed, and no extra stress environment is generated because a sentence requests it. These phrases should be read as attempted guidance whose effective scope is limited by the available interface. They must not be described as completed experiments.

The same principle applies to future agents. Giving an agent a role title or a sophisticated prompt does not establish that it has reliable information, usable tools, or authority to act. Capabilities are created through executable interfaces and observable behavior. This is a constructive design lesson: natural-language proposals can help identify desirable improvements, but each proposed improvement must be translated into a testable mechanism. Once implemented, its contribution should be measured against the simpler system it is intended to improve.

15 What the saved notebook run actually shows

The attached notebook contains stored outputs from a completed run [3]. Those outputs provide evidence about that recorded execution, not a distribution of outcomes across repeated independent runs. The initial policy has average validation utility approximately 0.989305. The seed improver, I_0 , returns the same score. After the first outer selection, the winning improver uses four candidates per round, two rounds, and two survivors, with an explore-then-exploit search instruction. Its discovered policy reaches validation utility 0.993648. The second outer selection retains the same specification and the same recorded score.

Generation	Validation utility	Held-out utility	Held-out RMSE	Held-out direction accuracy
I_0	0.989305	0.996561	0.011563	54.85%
I_1	0.993648	0.981650	0.011757	47.88%
I_2	0.993648	0.981650	0.011757	47.88%

Table 1: Values transcribed from the attached notebook’s saved history and transfer summary. Validation aggregates four training markets; held-out values aggregate three additional markets. RMSE is measured in log-return units. The generation labels refer to the policy discovered by each improver.

The first conclusion is that the configuration search changed the selected research procedure and improved its score on the selection sample. The absolute increase is 0.004343 utility units. The second conclusion is equally important: the improved score remains below one, so it does not beat the zero-return baseline on average. Indeed, the individual-model scorecard places the zero-return forecast first with utility 1.000000. Since the policy representation permits a single model in each block, an all-zero policy is admissible. The search therefore failed to reach an available benchmark that was visible in its own evidence.

This result does not make the exercise useless. It makes the lesson more precise. The inner loop preserves the best candidate it has encountered, not every candidate that could have been constructed. It does not automatically insert every individual model as a standalone policy into its initial population. An LLM may favor elaborate or family-diverse proposals even when a simple choice has the strongest measured support. A straightforward next version could include the zero policy and other elementary baselines explicitly, ensuring that a restricted search cannot overlook them merely because they were never proposed.

The third conclusion concerns transfer. Held-out utility falls from 0.996561 for I_0 ’s policy to 0.981650 for I_1 ’s policy, and held-out RMSE rises. Directional accuracy also falls. Thus, the recorded validation improvement is not accompanied by improved held-out forecasting performance. This is consistent with selection overfitting or sample-specific gains, but the saved run alone cannot identify a unique cause. It provides no basis for claiming profitable trading, statistically established predictive superiority, or a generally superior improver.

The plateau at I_2 is also informative. A new generation label need not mean a new capability. In this run, the incumbent wins again. The audit output records fourteen successful API calls and two outer selection events. These observations show that the bounded process executed and made inspectable choices. The central teaching achievement is an observable search-and-evaluation architecture whose results can be criticized quantitatively, including when its apparent improvement fails to transfer.

16 Reading the final policy and the diagnostic plots

The saved final policy illustrates conditional allocation among models. Its high-volatility block places weights 0.35, 0.25, 0.20, and 0.20 on the zero forecast, Huber regression, the twenty-observation rolling mean, and the historical mean. This is a relatively conservative mixture in the limited sense that part of its forecast is explicitly shrunk toward zero. Its positive-trend block instead combines linear regression, AR(1), random forest, and support vector regression. The other blocks use different mixtures. These are concrete configurations students can inspect, reproduce, and compare with simpler alternatives.

A coherent description of those weights is not an explanation established by evidence. We may hypothesize that the high-volatility block reduces excessive reactions to noisy observations. To demonstrate that mechanism, we would need a controlled comparison removing or changing the zero component while holding other conditions fixed. The same applies to the positive-trend mixture. A narrative about complementary models is useful as a research hypothesis, but the saved overall score does not prove that each component contributes positively. Ablation experiments would turn the narrative into a testable account.

The price plot requires particular care. The notebook converts each one-step log-return forecast into a price using the observed current price multiplied by the exponential of the predicted return. It plots that value against the actual next-day price. These are successive one-step forecasts anchored to observed prices, not a freely evolving multi-step forecast path. Even a zero-return forecast follows the current price closely and may look visually impressive when prices are persistent. A close match between two price lines is therefore much weaker evidence than an improvement in return prediction error against the baseline.

Regime diagnostics use the generator's hidden state after forecasts have been formed. They do not feed that state into the policy. The saved final-policy diagnostic reports mean utility about 0.9676 in the bearish high-volatility state and 1.0100 in recovery momentum, with other states near but below one. These are subgroup summaries, subject to limited sample sizes and the notebook's aggregation rule. The stored label is the state at the prediction row, whereas the target return belongs to the following observation. Because state transitions can occur, this diagnostic should not be described as exact conditioning on the next return's generating state.

Together, the tables, weights, and plots provide different views of the same experiment. The history describes search progress; the final policy describes the selected rule; the price plot illustrates its application; and subgroup summaries suggest where to investigate failures. No single view substitutes for the others. A convincing research account connects them while respecting what each can establish. That interpretive discipline is one of the most transferable skills students can learn from the notebook.

17 Why self-improvement needs independent evidence

An optimizer repeatedly observes the consequences of its choices on the same selection sample. Over time, that sample becomes part of the system's effective information. Even when individual model forecasts are generated without looking into the future, the policy search can adapt to peculiarities of the validation observations. Chronological correctness in base-model fitting and independence in policy evaluation are therefore separate requirements. The first prevents forecasts from using information that was unavailable at decision time. The second asks whether the selection process has become too specialized to the evidence used to choose it.

The outer loop increases the number of choices made using validation evidence. It compares not only policies but also the procedures that produce policies. This does not invalidate the architecture; it increases the importance of a disciplined evaluation design. The selected outer configuration has benefited from a search over search procedures, and its reported validation score should be interpreted accordingly. A higher value is evidence that the procedure found a better candidate on those observations. It is not an unbiased estimate of future forecasting quality.

The notebook's test stage offers a useful boundary by applying selected policies to later observations in three additional simulated histories. However, if a researcher repeatedly examines those test results and adjusts the notebook in response, that test set gradually becomes development information. A subsequent version should use fresh environments or another reserved evaluation layer for its final assessment. The boundary is defined by how data are used, not by a variable name such as `TEST_MARKETS`. Once outcomes influence design choices, their role in the research process has changed.

A stronger experiment could compare complete pipelines over repeated market seeds and repeated proposal runs. It could also distinguish policy transfer from improver transfer explicitly. For the former, freeze a discovered policy and evaluate it on fresh observations. For the latter, freeze the improver specification, allow it to search using a new task's development data, and evaluate the newly discovered policy on that task's untouched evaluation data. These experiments answer different questions. The current notebook implements the former, while the latter would more directly test whether a reusable search method has been learned.

Students should also compare the LLM-guided search with simpler procedures under matched budgets. A random sampler of admissible policies, a fixed equal-weight ensemble, and an explicit baseline population would help determine whether language-model proposals add value beyond search itself. Without those comparisons, an improvement over the initial policy cannot be attributed uniquely to the LLM. This is not a reason to abandon the architecture. It is the research design required to learn which components deserve to remain when the system becomes more sophisticated.

18 A more informative inner loop

The next version need not begin by adding more agents. It could begin by improving the evidence supplied to the LLM already present in the inner loop. After each evaluation, Python could return a compact diagnostic report describing utility by market, error by observable regime, prediction scale, and the relationship between component errors. The LLM would receive the current parent together with that report and a specific question about what to test next. This would turn selection-mediated feedback into an explicit cycle of proposal, measurement, diagnosis, and revision.

Suppose a candidate performs well in calm observations but makes unusually large errors during volatile periods. A diagnostic prompt could ask whether the next experiment should reduce a particular weight, increase shrinkage toward zero, or compare a simpler high-volatility block. The resulting proposal would still be represented in a bounded schema. Python would evaluate it using the same protocol and record whether the expected improvement occurred. The LLM's explanation would help formulate a hypothesis, but the hypothesis would remain separate from the measured outcome.

A useful extension would require each proposal to include an expected mechanism and a comparison plan in addition to its policy fields. These extra fields would be research metadata, not executable authority. For example, a proposal might state that it changes only one regime block to isolate the effect of removing a redundant component. The evaluator could verify whether only that block changed and then report the difference against the parent. This would make the search history more intelligible and reduce the temptation to explain a large bundle of simultaneous changes after seeing a favorable

score.

The system could also maintain a compact experiment ledger. A ledger entry would connect the parent, candidate, changed fields, evaluation dataset signature, score, and diagnostic observations. When generating later proposals, the LLM could be shown relevant failed experiments as well as successes. This is more useful than repeatedly asking for novelty without defining what has already been tried. Yet the ledger should preserve uncertainty: a candidate that failed on one sample may still represent a plausible idea, and a small positive result may be unstable. Memory should support better experiments rather than turn accidental outcomes into permanent beliefs.

Finally, a richer inner loop could stop when additional proposals no longer justify their cost. That would require an explicit stopping criterion based on remaining budget, recent gains, or uncertainty, rather than an instruction to be efficient. The current fixed round count is transparent and appropriate for a first notebook. A future adaptive stopping rule would itself become an object of comparison. The guiding principle is incremental sophistication: add one measurable capability, identify the question it answers, and test whether it improves the complete research process.

19 Making the outer loop more discriminating

The present outer score assigns each improver the utility of the best policy found in one search. A more discriminating evaluator could run each specification several times with controlled variation in proposals and environments. Instead of rewarding only the most favorable discovery, it could assess average performance, dispersion, failure rate, and computational cost. This would help distinguish a reliably useful recipe from one that occasionally obtains a high score. The appropriate aggregation depends on the teaching or research objective, but it should be specified before inspecting the candidate rankings.

Budget matching is especially important. One possible comparison grants each recipe the same number of proposal tokens or candidate evaluations and allows it to allocate that budget differently. Another comparison measures how much score improvement each recipe delivers at several fixed budget levels. The first asks which organization of a given resource is better. The second reveals whether a method is attractive only when substantial computation is available. Neither question is answered fully by permitting more rounds and then ranking solely by the best resulting score.

The configuration space could become more expressive without permitting arbitrary code execution. New fields might specify a schedule for candidate generation, a numerical diversity threshold, a stagnation rule, or a choice among approved diagnostic tools. Each field would need a defined implementation and validation rule. A request for “broad exploration” could then map to a concrete sampling constraint, and a request for “stress evaluation” could invoke a particular set of simulated scenarios. The advantage is that a proposal becomes both more capable and more auditable.

More direct program improvement would constitute a further step toward the original STOP formulation. The system could propose changes to an approved search module, run them in an isolated evaluation environment, and compare results under fixed tests. This would substantially expand the search space and the verification burden. Correct execution, permitted resource use, and compatibility with the evaluator would become part of acceptance. It should therefore be introduced as a new research stage, not described as a small adjustment already present in the configuration-only notebook.

A mature outer loop would preserve the distinction between proposing evaluation changes and choosing candidates under the current evaluator. If a recipe could alter the rule by which it is judged during the same contest, a higher score might reflect an easier test rather than a better search. Evaluation design

can evolve, but changes should be versioned and compared openly. The central challenge is to make the system better at experimentation without allowing it to erase the meaning of the experiments used to establish improvement.

20 From forecasts to trading strategies

Replacing forecasting models with trading strategies changes the object being optimized. A forecasting model maps information to an estimate of a future quantity. A trading strategy maps information and portfolio state to intended actions over time. It must address entry, exit, position size, exposure limits, and the conditions under which it refrains from trading. Two strategies using the same forecast can produce very different outcomes because they trade at different thresholds, hold positions for different periods, or incur different costs. The forecast-selection notebook therefore supplies an architectural foundation rather than a completed trading system.

A strategy library could contain trend-following rules, mean-reversion rules, volatility-targeting allocations, relative-value positions, and event-driven approaches. Each strategy would have a defined parameter space and a common interface to the simulator. The inner loop could propose parameter configurations, combinations of strategies, or rules for allocating capital among them. The outer loop could compare the procedures used to discover those configurations. The nesting remains recognizable, but the evaluator must now simulate stateful decisions and their consequences rather than simply combine columns of return forecasts.

Portfolio interactions make the extension richer. Combining two standalone strategy return series is not always equivalent to running their combined orders in one account. Strategies may compete for capital, offset positions, concentrate in the same exposures, or trade simultaneously when liquidity is scarce. Their marginal costs can depend on total turnover and order size. A credible combined-strategy evaluator therefore needs shared portfolio state and a coherent execution model. Reusing cached signals may still be valuable, but blindly averaging independently backtested profits can miss the interactions that matter most.

The utility function must also change. A trading objective might reward net performance while penalizing drawdown, turnover, or concentration, subject to hard exposure limits. These terms should be interpretable and consistently scaled. Some requirements are better treated as admissibility constraints than as penalties that a large return can offset. For example, a configuration outside a stated leverage limit can be rejected before its performance is ranked. The evaluator should make that distinction explicit rather than hide every concern inside one opaque score.

Such a strategy system should be tested in stages. Historical simulation establishes whether the logic behaves as intended under specified assumptions. Fresh evaluation periods test selection transfer. A forward paper-trading stage tests data timing and operational behavior without assuming that historical fills can be reproduced. The point is not to add procedural complexity for its own sake. It is to align the meaning of “improvement” with the more demanding object now being optimized. A better forecast score is a useful educational milestone; a better trading strategy requires a broader account of decisions, costs, and risk.

21 Strategies represented by LLM-based agents

An LLM-based strategy agent can be understood as a decision procedure that uses language-model reasoning together with tools, information, and constraints. It might interpret a macroeconomic

release, compare a company announcement with an existing thesis, or decide which quantitative signal deserves further investigation. Its output could be a structured signal, a proposed target position, or a request for another calculation. To evaluate it as a strategy component, however, we must define its operating contract. A role description alone is insufficient.

The contract should specify the available information, the decision time, the tools the agent can call, the permitted output fields, and how long the proposal remains valid. A research agent might be allowed to request a volatility estimate but not to place an order. A strategy agent might propose a target exposure but rely on a separate deterministic layer to enforce account limits. These distinctions make the behavior measurable. They also allow students to distinguish a language model expressing an opinion from a system producing an operationally interpretable decision.

Historical evaluation introduces an additional problem when the LLM may have encountered later information during training. Supplying only documents dated before a simulated decision does not necessarily remove information already present in the model's parameters. Fresh synthetic tasks, carefully anonymized experiments where appropriate, and forward evaluation can help investigate this issue, although none should be treated as a universal solution. The current notebook avoids direct historical interpretation by using the LLM mainly to select generic forecasting configurations for synthetic data. An event-reading strategy agent would face a different evidentiary challenge.

Reproducibility also becomes more demanding if the agent is queried repeatedly during a backtest. Its outputs may vary across runs or service versions, and latency may affect when a decision can be acted upon. A useful experiment would retain the exact input package, tool results, structured response, model identifier, and decision timestamp for every action. It could distinguish replaying a recorded decision sequence from regenerating the agent's decisions. Those are different experiments: replay measures downstream execution under fixed decisions, while regeneration measures the stability of the decision procedure itself.

The inner loop could then compare agent specifications as candidate strategies. It might vary the permitted tools, information summary, decision cadence, or numerical thresholds while keeping the evaluation contract fixed. A candidate would earn acceptance through outcomes and compliance with that contract, not through more elaborate reasoning text. This preserves the central lesson of the forecasting notebook as the representation becomes richer: the LLM can propose and interpret, but useful adaptation must become observable in a controlled sequence of actions and measured consequences.

22 Agents as an organized research process

Multiple agents become useful when distinct responsibilities require different information, tools, or forms of scrutiny. A data agent could verify timestamps and dataset consistency. A quantitative research agent could propose strategy variants and request numerical experiments. A diagnostic agent could examine residuals, unstable exposures, and failure cases. A risk agent could test a candidate against defined constraints. An execution agent could translate an accepted portfolio target into an approved order plan. Their value would arise from what each can actually verify or produce, not from the number of agents in the diagram.

A coordinating agent could organize these contributions through explicit work requests. Suppose a candidate strategy earns a higher net return but doubles turnover. The coordinator might ask the execution evaluator to estimate sensitivity to higher costs, the risk evaluator to inspect concentration, and the research agent to propose a lower-turnover variant. Each response would enter a shared experiment record. The coordinator would not need to decide which narrative sounds most confident;

it would need to determine whether the required evidence exists and whether the candidate passes the stated acceptance rule.

Specialization does not automatically create independence. Several agents built on the same language model and supplied with the same summary can repeat the same error. A critic is more useful when it can inspect primary data, run a separate calculation, or challenge a specific assumption. Disagreement should lead to a resolvable test where possible. For example, an objection about look-ahead information can be investigated through timestamps and data lineage. A disagreement about cost sensitivity can be tested through a simulator. More text exchanged between agents is not a substitute for discriminating evidence.

An outer loop could eventually compare organizational arrangements. It might test whether diagnostics should precede expensive backtests, whether a specialist should review only candidates above a threshold, or whether a larger diversity of research hypotheses justifies added cost. Such an experiment would optimize the research organization under a common task and resource envelope. The allowed organizational changes would still need a representation and an evaluator. Otherwise, the system would merely generate descriptions of better teamwork without measuring their operational consequences.

This extension also clarifies where human judgment remains essential. The choice of objective, admissible risk, deployment scope, and interpretation of uncertain evidence defines the research mandate. Agents can help investigate those choices and execute approved procedures, but the system should not silently redefine them in pursuit of a higher internal score. A capable research organization combines exploration with accountability: it knows what changed, why the change was proposed, how it was tested, and what evidence would justify reversing it.

23 A progression for classroom experiments

A productive classroom sequence begins with understanding the existing result before expanding the architecture. Students can first explain why an improved validation score remains below the baseline, then construct the admissible all-zero policy and compare it with the recorded winner. This exercise requires almost no new machinery, yet it exposes the difference between the best candidate found and the best candidate available. It also teaches that baselines belong inside the experiment's acceptance logic when they represent viable operational alternatives.

The next exercise can isolate one source of search value. Students might compare LLM-generated candidates with randomly generated valid policies under the same number of candidate evaluations. Alternatively, they could hold the candidate policies fixed and compare survivor counts, or hold the budget fixed and compare shallow broad search with deeper narrow search. Each exercise should change one meaningful dimension. The purpose is to identify mechanisms rather than accumulate a collection of impressive but confounded improvements.

A third exercise can add a diagnostic report to the inner prompt and test whether it helps. The report should be generated from the same development observations already available to the search, leaving the evaluation boundary intact. Students would compare proposal quality, repeated failures, and final performance rather than judging the explanatory fluency of the LLM's response. A useful negative result would be that richer diagnostics increase token consumption without improving outcomes. The experiment would then have identified a capability that does not yet justify its cost.

Only after these comparisons should the class replace forecasts with a small strategy library. A minimal strategy can use a threshold on a signal, a fixed position-sizing rule, and an explicit cost assumption. Students can then observe why a lower forecast error does not necessarily produce a

higher net return. Subsequent versions can introduce shared portfolio state, adaptive exposure, and an agent that interprets structured research. This progression preserves conceptual continuity while adding one new source of difficulty at a time.

By the end, students should be able to describe the system without relying on the word “autonomous” as an explanation. They should identify the candidate object, the proposal mechanism, the allowed changes, the evaluator, the selection rule, and the evidence of transfer. Those questions apply equally to a forecasting ensemble, a strategy agent, and a coordinated research organization. The notebook is therefore a foundation for learning how to build and interrogate adaptive systems, rather than a demonstration to be admired only when its scores rise.

24 Conclusion: the discipline of improving improvement

The most important transition in this paper is from treating intelligence as a property of one model to studying the behavior of a complete learning process. Familiar statistical methods remain useful. The language model adds a way to propose configurations, while Python supplies repeatable numerical evaluation and selection. The inner loop searches for a better forecasting policy. The outer loop searches for a better configuration of that search. The resulting architecture makes improvement an observable sequence of proposals, tests, and retained decisions.

The current implementation is deliberately bounded. Forecasting models are fixed during policy search, regime thresholds are fixed, policy responses are structured data, and the outer layer changes a small improver specification. The LLM participates in both loops but does not retrain itself or rewrite the executable program. These boundaries help students understand exactly what adapts. They also reveal the next engineering questions: which additional evidence would improve proposals, which new configuration fields deserve implementation, and how should a more capable search be evaluated fairly?

The saved run provides a particularly valuable lesson because it resists a simplistic success story. The first outer selection raises validation utility, but the discovered policy remains below the zero-return benchmark and performs worse on held-out observations. The second selection preserves the same configuration. The architecture works as an inspectable experiment, while its recorded results do not establish predictive superiority. Recognizing both facts is the appropriate scientific response. A system that makes its limitations visible is a better teaching instrument than one whose complexity obscures what happened.

Future versions can expand from forecasting policies to complete algorithmic trading strategies, then to agents that interpret information and coordinate research. Each expansion changes the meaning of the candidate, the cost of evaluating it, and the evidence required for acceptance. Strategy utilities must account for decisions and costs; agent evaluations must account for information access, reproducibility, and operational contracts. Organizational search must demonstrate that additional roles improve results under a clear mandate rather than merely increase the volume of deliberation.

The principle that survives all these extensions is straightforward: a system can search over solutions, and it can search over the procedures used to find those solutions. What makes that principle valuable is the quality of the connection between adaptation and evidence. Students who learn to design that connection have moved beyond choosing among forecasting algorithms. They have begun to build research systems capable of proposing changes, measuring their consequences, preserving what works, and recognizing when an apparent improvement should not survive the next test.

A Connecting the narrative to the notebook

The notebook uses numbered code sections from 0 through 34, together with an additional empty code cell in the supplied file. The numbering below follows the section comments rather than absolute notebook cell positions, because pedagogical Markdown cells are interleaved with code. This map is intended to help a reader move from the conceptual discussion to the implementation without turning the paper into a line-by-line programming manual.

Code sections	Responsibility	What to inspect
0–3	Environment and settings	Dependencies, seeds, model identifier, API budget, refit interval, secret loading.
4–5	Synthetic data and features	Hidden states, observable information, shifted targets, cleaned row alignment.
6–9	Base forecasting layer	Model menu, implementations, fallbacks, walk-forward tables, cache signature.
10–16	Policy evaluation	RMSE utility, regime proxy, policy blocks, initial policy, aggregation, score-card.
17–21	Inner search	API cache, JSON response, specification, candidate generation, survivor selection.
22–25	Outer search	Proposed specifications, inner trials, meta-score, winner, generation record.
26–28	Experiment and histories	Initial score, complete run, outer plot, reconstructed inner histories.
29–33	Evaluation and inspection	Held-out policies, one-step price plot, regime diagnostics, final weights, audit summary.
34	Architecture summary	Names and responsibilities of the main system components.

Table 2: A navigation map for the supplied notebook. This paper preserves and describes its implementation; it does not modify or rerun it.

The inner-history visualization invokes the saved improver specifications again rather than simply displaying histories retained by the outer routine. In the current session, identical prompts can reuse cached responses, making those reconstructed traces stable under the same conditions. A future version could retain each original inner history directly with its outer evaluation, removing ambiguity between a recorded run and a reconstruction. This is a small example of how data structures can improve the auditability of a learning system.

The configured language-model identifier is recorded as `claude-sonnet-5` in the attachment. It is treated here as a property of the supplied artifact, not as a verified statement of current service availability. The architectural argument does not depend on that specific identifier. The relevant role is a fixed language model accessed through a bounded proposal interface, with numerical evaluation performed outside it.

B Source notes

The notebook is the primary source for all implementation claims and saved-run numerical values in this paper. The supplied eight-page teaching note provides the pedagogical framing around scaffolds, improvers, and meta-utility. The original STOP reference establishes the research lineage. Proposed extensions in the later sections are design directions developed here; they are not reported features of the notebook or empirical findings of the original research.

The LaTeX source follows the supplied visual sample's navy, blue, and teal palette, colored rules, restrained boxes, and running author information. Body typography has been expanded for a longer teaching paper. This version is fully text based and does not load external image files or reserve locations for image placeholders.

References

- [1] Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. *Self-Taught Optimizer (STOP): Recursively Self-Improving Code Generation*. Conference on Language Modeling, 2024. arXiv:2310.02304, version 3. <https://arxiv.org/abs/2310.02304>.
- [2] Alejandro Reynoso. *When the Improver Improves Itself: A Pedagogical Introduction to the Self-Taught Optimizer (STOP)*. September 2026. Supplied course teaching note, eight pages; attachment: `STOP_Pedagogical_Teaching_Note(2).pdf`.
- [3] *TOPIC 768.2: STOP Algorithmic Trading Pedagogical Model*. Supplied Google Colab notebook, September 2026. Attachment: `TOPIC_768_2_STOP_ALGO_TRADING_PEDAGOGICAL_MODEL.ipynb`. Code sections 0–34 and saved outputs; primary implementation and numerical source for this paper.